American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science &Technology (FST)

Spring 24 25

Section: B/ C

Software Quality Assurance and Testing

PROJECT TITLE

A Report submitted
By

SN	Student Name	Student ID
1		
2		
3		
4		

Checked By Industry Personnel (Optional)

Name:

Designation:

Company:

Sign:

Date:

Software Test Plan

for

AutoCart: Intelligent E-Commerce with Predictive Auto-Refill & Smart Bundling

Version 1.0 approved

Prepared by <author>

<organization>

<date created>

Table of Contents

Revision History

Revision	Date	Updated by	Update Comments
0.1	2007.06.04	Scot Robinson	First Draft
0.2	2007.06.19	Amit Nimse	

1. TEST PLAN IDENTIFIER: TP-AutoCart-2025-001

- **TP** → Test Plan
- **AutoCart** → Name of the project (Intelligent E-Commerce with Predictive Auto-Refill & Smart Bundling)
- **2025** → Year of execution and testing
- **001** → First official version of this test plan

This test plan covers the Software Quality Assurance and Testing strategy for **AutoCart**, an intelligent e-commerce platform designed to automatically predict product refill needs and offer smart bundle suggestions to users. The identifier serves as a unique reference for Version control of the test documentation ,Traceability across all QA reports and deliverables, Coordination among team members, especially during updates or revisions

All future updates to the test plan will follow a consistent versioning format (e.g., TP-AutoCart-2025-002 for version 2, and so on).

2. REFERENCES

- **Academic Course Materials** – Lecture slides, class notes, and examples from the Software Quality Assurance and Testing (SQAT) course provided foundational knowledge for test design, planning, and execution.
- **Figma**– Used for designing the User Interface (UI), creating wireframes, and prototyping the AutoCart platform to ensure a seamless and user-friendly experience.
- **Software Requirements Specification (SRS)** – A detailed documentation of system requirements (both functional and non-functional) was prepared to define the testing scope and to validate expected outcomes.
- **GitHub Repository** – The central codebase of AutoCart, version-controlled using GitHub, served as a reference for feature implementation and tracking tested modules.
- **Third-Party Libraries and APIs Documentation** – Referred to documentation for tools like Firebase, Stripe (for mock payments), and Framer Motion used in frontend development, to ensure correct test coverage and behavior.
- **Industry Standards and Best Practices** – Testing approach was guided by international standards such as:
    1. ISO/IEC 25010 – For defining quality characteristics (like usability, reliability, etc.)
    2. IEEE 829 – Standard for Software Test Documentation
    3. General QA practices for e-commerce platform
- **Online Research & Articles** – Consulted blogs, research articles, and case studies related to subscription-based e-commerce, predictive ordering, and smart bundling to guide the testing design and ensure relevance to real-world practices.

3. INTRODUCTION

AutoCart: Intelligent E-Commerce with Predictive Auto-Refill & Smart Bundling is a next-generation subscription-based e-commerce platform built to automate the replenishment of essential household products. It aims to reduce user effort, minimize waste, and enhance shopping efficiency through intelligent refill prediction and smart bundling features.

With AutoCart, users can subscribe to items like toiletries, groceries, or supplements and receive refill reminders or automatic deliveries based on usage patterns. By integrating predictive logic, personalized recommendations, and eco-conscious design, AutoCart elevates the standard e-commerce model into a smart, user-friendly solution.

Background to the Problem

In today's fast-paced world, many individuals face recurring problems related to managing household essentials:

- **Forgetting to Reorder Essentials** – Busy schedules cause users to run out of daily-use items like toothpaste, detergent, or protein powder.
- **Manual and Repetitive Shopp**ing – Users waste time reordering the same items manually every month.
- **Poor Personalization in Subscription Models** – Traditional subscription systems are rigid and don't adapt to actual consumption behavior.
- **Missed Savings Opportunities** – Customers often miss discounts or fail to optimize shipping by combining refills.
- **Limited Focus on Sustainability** – Current platforms rarely help users minimize packaging waste or carbon impact.

The **root cause** of this problem is the lack of dynamic, usage-aware logic in current subscription models, which operate on fixed intervals instead of actual consumer behavior. Despite the rise of subscription e-commerce, most systems are static—offering fixed intervals instead of intelligent, usage-based predictions. This results in unnecessary orders or product shortages, lowering user satisfaction and contributing to environmental waste.

Solution to the Problem

AutoCart is designed to address these challenges through an AI-assisted refill automation and bundling system. Key solutions include:

- **Smart Refill Prediction Engine** – Tracks previous order frequency and consumption data to intelligently recommend optimal refill times.
- **Personalized Subscription Management** – Users can pause, resume, or adjust refills with one click based on actual usage.
- **Smart Bundling System** – AutoCart identifies multiple products likely needing refills together and offers bundle discounts with optimized shipping.

- **Sustainability Dashboard** – Shows users the environmental impact of bundled shipments and refill frequency, promoting eco-friendly behavior. Sustainability Dashboard – Shows users the environmental impact of bundled shipments and refill frequency, promoting eco-friendly behavior.
- **User-Centric Dashboard** – Centralized interface for managing all subscriptions, refill statuses, preferences, and offers.

By merging personalization, automation, and sustainability, AutoCart transforms e-commerce into a proactive experience—saving time, minimizing waste, and ensuring customers never run out of what they need.

4. REQUEIREMNT SPECIFICATION

4.1 System Features

1. Sign Up / Registration

Functional Requirements:

1.1 The system shall allow users to register using a valid email address, phone number, and secure password.

1.2 The system shall validate input fields such as email format and password strength during sign-up.

1.3 The system shall prevent duplicate registrations using the same email or phone.

Priority Level: High
Precondition: User must have a valid email address or phone number.

2. User Authentication

Functional Requirements:

2.1 The system shall allow users to log in using their registered email or phone and password.

2.2 The system shall provide a “Forgot Password” option to reset credentials via OTP or email link.

2.3 The system shall lock the account for 30 minutes after 5 consecutive failed login attempts (optional).

Priority Level: High
Precondition: User account must be registered and verified.

3. Subscription Management

Functional Requirements:

2.1 The system shall allow users to subscribe to one or more products from the catalog.

2.2 The system shall allow users to pause, resume, or cancel their product subscriptions.

2.3 The system shall show current subscription status (active, paused, canceled) in the dashboard.

Priority Level: High
Precondition: User must be logged in with a verified account.

4. Smart Refill Prediction Engine

Functional Requirements:

3.1 The system shall track product usage frequency based on previous orders.

3.2 The system shall predict the next refill date based on average consumption intervals.

3.3 The system shall notify users with refill suggestions before stock runs out.

Priority Level: High

Precondition: User must have at least one product subscribed and previous order history.

5. Smart Bundling System

Functional Requirements:

4.1 The system shall analyze multiple products with overlapping refill windows.

4.2 The system shall suggest bundled orders to minimize shipping and maximize savings.

4.3 The system shall apply bundle discounts automatically during checkout.

Priority Level: High

Precondition: User must have more than one active product subscription.

6. Product Catalog & Recommendations

Functional Requirements:

5.1 The system shall allow users to browse and search for products.

5.2 The system shall recommend products based on past purchases and category interest.

5.3 The system shall display detailed product info (price, description, reviews).

Priority Level: Medium

Precondition: User must be logged in to view personalized recommendations.

7. Checkout & Payment System

Functional Requirements:

6.1 The system shall allow users to review order summary before checkout.

6.2 The system shall support payments through debit/credit cards and mobile banking.

6.3 The system shall confirm successful payments and generate order receipts.

Priority Level: High

Precondition: User must be logged in and have valid payment credentials.

8. Notification System

Functional Requirements:

7.1 The system shall send timely refill reminders via email or in-app alerts.

7.2 The system shall notify users of bundle opportunities and ongoing offers.

7.3 The system shall allow users to customize notification preferences.

Priority Level: Medium

Precondition: User must be logged in and have active subscriptions.

9. Sustainability Dashboard

Functional Requirements:

8.1 The system shall track and display statistics on reduced packaging and shipment frequency.

8.2 The system shall compare single vs. bundled orders in terms of carbon footprint.

8.3 The system shall encourage users with eco-friendly suggestions.

Priority Level: Low

Precondition: User must have completed at least one bundled order.

10. Order Tracking System

Functional Requirements:

9.1 The system shall show real-time order status (processing, shipped, delivered).

9.2 The system shall provide estimated delivery date and shipment tracking info.

Priority Level: Medium

Precondition: User must have placed an order.

4.2 System Quality Attributes

1. Performance

- **Requirements:**
  - The system shall respond to user actions (e.g., adding to cart, modifying subscriptions) within **2 seconds**.
  - The system shall process checkout and bundling logic in under **3 seconds**.
  - The platform shall support up to **5,000 concurrent users** during promotional sales without performance degradation.
- **Priority:** High
- **Measure:** Response time, system load under concurrent users
- **Description:** AutoCart should perform efficiently to maintain a seamless user experience even during peak shopping times.

2. Scalability

- **Requirements:**
  - The system architecture shall support **horizontal scaling** using cloud services.
  - The system shall scale to support **up to 100,000 registered users** and growing inventory without re-architecture.
- **Priority:** High
- **Measure:** Number of concurrent users, system resource usage, load distribution
- **Description:** AutoCart should be designed to accommodate increasing users and product data as the platform grows.

3. Usability

- **Requirements:**
  - The system shall feature a **user-friendly dashboard** for managing subscriptions, bundles, and delivery preferences.
  - The platform shall provide **onboarding guidance, tooltips**, and a searchable **help section**.
  - Users should be able to perform key actions (like pausing a subscription) in **3 clicks or fewer**.
- **Priority:** Medium
- **Measure:** Task completion time, user feedback, support request volume

- **Description:** AutoCart should be intuitive for both tech-savvy and non-technical users to encourage adoption and reduce support burden.

4. Reliability

- **Requirements:**
  - The system shall maintain an **uptime of at least 99.9%**, especially during refill reminder periods and sales.
  - In case of server failure, the system shall **auto-recover within 3 minutes** using fallback containers or replicas.
- **Priority:** High
- **Measure:** Uptime logs, failover testing reports, MTTR (Mean Time to Recovery)
- **Description:** The system should be dependable and available to prevent missed refills or failed transactions.

5. Security

- **Requirements:**
  - All user credentials and payment data shall be stored using **end-to-end encryption**.
  - The system shall use **token-based authentication** with optional **multi-factor authentication** for sensitive actions.
  - The system shall log and monitor all critical actions for **auditability and fraud detection**.
- **Priority:** High
- **Measure:** Number of vulnerabilities, security breach logs, audit trail completeness
- **Description:** User data must be fully protected to ensure trust, privacy, and compliance with security best practices.

4.3 System Interface

- Draw the system interface where the users will interact with the system's functionality.
  (At least 12 page figma ui design must be provided)

4.4 Project Requirements

4.4.1 Budget Constraints

The overall project budget is estimated at 4,25,000 BDT, which includes costs for development, infrastructure, security, marketing, and post-launch support

Category	**Estimated Cost (BDT)**
AI Refill Prediction Engine Development	1,00,000
Software Development (Frontend & Backend)	90,000
Cloud Infrastructure (Hosting, DB, Storage)	60,000
Third-Party API Integration (Payments, Auth)	40,000
Subscription & Bundling Logic Implementation	30,000
Testing & Quality Assurance (QA)	30,000
Post-Launch Maintenance & Updates	30,000
Marketing & User Onboarding Materials	20,000
Security & Compliance (SSL, MFA, etc.)	25,000
Total Budget	**4,25,000 BDT**

4.4.2 Time Constraints

The projected development timeline for AutoCart is approximately 5 months, segmented into key phases such as planning, design, development, integration, and final deployment.

Phase	**Estimated Duration**
Requirement Analysis & Planning	2 weeks
UI/UX Design & Wireframing	3 weeks
AI Engine Development (Prediction)	1 month
Backend Development (APIs & DB)	1 month
Frontend Development (UI Logic)	1 month
Subscription & Cart Logic Setup	2 weeks

Integration, Testing & Deployment	2 weeks
Total Estimated Time	**5 months**

4.4.3 Resource Constraints

The successful execution of the AutoCart project depends on the availability of specific human and technological resources to ensure timely and efficient development.

Human Resources

- **Project Manager (1)** – Oversees project timelines, team coordination, and milestone tracking.
- **AI/ML Engineers (2)** – Develop and train the predictive refill engine and smart bundling algorithms.
- **Backend Developers (2)** – Implement server-side logic, subscription management, and database operations.
- **Frontend Developers (2)** – Build responsive and interactive user interfaces for web and mobile platforms.
- **QA Engineers (2)** – Conduct functionality, usability, and performance testing.
- **UI/UX Designer (1)** – Designs intuitive user flows and enhances customer experience.
- **Marketing Strategist (1)** – Plans go-to-market campaigns and handles pre-launch promotions.

5. Technology Resources

- **Frontend Development:** Next.js (React-based), Tailwind CSS
- **Backend Development:** Node.js with Express.js or Next.js API routes
- **AI/ML Libraries:** Python (scikit-learn, TensorFlow), Time-series analysis tools
- **Database:** PostgreSQL (relational data), Redis (caching), MongoDB (optional for dynamic content)
- **Cloud Services:** Vercel (frontend hosting), AWS/GCP (backend, storage, database)
- **Security Tools:** SSL, OAuth 2.0, Two-Factor Authentication (2FA), Data Encryption at Rest and

6. FEATURES NOT TO BE TESTED

Third-Party Application Integration & Data Usage

While AutoCart enables users to export their subscription and purchase history for personal use or integration with third-party budgeting or tracking tools, the accuracy, compatibility, and performance of such data within external applications are outside the scope of this project.

AutoCart will ensure proper data export formatting, but testing and maintenance of features within external tools remain the responsibility of those respective application developers or maintainers.

7. TESTING APPROACH

7.1 Testing Levels

- **Unit Testing**
  - Developers will perform unit testing for individual features such as smart refill prediction, product bundling, subscription management, and payment processing.
  - Each unit must include documented test cases, sample data, expected output, and defect reports before moving to the next testing phase.
  - Unit testing will be reviewed and approved by the development team lead to ensure all modules function independently.

- **System/Integration Testing**
  - A dedicated QA team will perform integration testing with assistance from developers.
  - Tests will validate interactions between modules (e.g., auto-refill triggers shipping, bundled checkout updates delivery schedule).
  - Integration with third-party services like payment gateways and delivery APIs will also be tested.
  - All major defects must be resolved, and critical functionality must pass validation before advancing to user acceptance.

- **User Acceptance Testing (UAT)**
  - Real users (test shoppers) will evaluate the full AutoCart workflow, including product selection, refill recommendations, checkout, and order tracking.
  - UAT will run in parallel with existing manual subscription methods (e.g., regular reminder emails) for a fixed period.
  - Feedback from UAT will be collected to identify usability gaps and minor bugs before official launch.

7.2 Test Tools

Testing for the AutoCart platform will utilize a combination of automated testing tools, performance testing frameworks, and bug tracking systems to ensure a robust and efficient QA process.

A. Automated Testing Tools

Visual Studio Code with Jest & React Testing Library

- **Purpose**: Unit and component testing for frontend features such as product cards, subscription flows, and dynamic refill alerts.
- **Usage**: Validate UI logic, interaction states, and conditional rendering in the Next.js-based frontend.
-

• Postman

- **Purpose**: API testing for endpoints including smart refill triggers, cart operations, and user subscription management.
- **Usage**: Validate request/response payloads, authentication flows, and error handling for backend APIs.

B. Performance Testing Tools

• Apache JMeter

- **Purpose**: Simulate user traffic and assess system performance under load.
- **Usage**: Test scenarios like peak-time cart checkouts, large bundle creation, and batch refill notifications.

C. Bug Tracking & Reporting Tool

• Trello or Jira

- **Purpose**: Track bugs, log defects, and organize the QA workflow.
- **Usage**: Assign issues to developers, prioritize critical bugs, and maintain visibility of testing progress across the team.

7.3 Meetings

- The QA team will conduct **weekly meetings** to assess testing progress, review identified defects, and ensure alignment with testing goals.
- **Bi-weekly coordination meetings** will be held between the QA lead, development team, and project manager to ensure smooth integration and timely resolution of cross-functional issues.
- **Ad-hoc/emergency meetings** may be scheduled when critical bugs or high-priority issues arise that require immediate attention and collaborative resolution.

8. TEST CASES/TEST ITEMS

(at least 15 test cases must be provided)

Project Name:			Test Designed by:	
Test Case ID: FR_1			Test Designed date:	
Test Priority (Low, Medium, High): Medium			Test Executed by:	
Module Name: Login Session			Test Execution date:	
Test Title: verify login with valid username and password				
Description: Test website login page				
Precondition (If any): User must have valid username and password				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Go to the website 2. Enter username 3. Enter password 4. Click submit	Username: 99999999999 Password: 321	User should login into the application	As expected,	Pass
Post Condition: User is validated with database and successfully login to account. The account session details are logged in the database.				

9. ITEM PASS/FAIL CRITERIA

The testing process for the AutoCart system will be considered complete when the following criteria are satisfied:

1. **Core Functionality Verification**: All essential features — including smart refill prediction, personalized bundle creation, subscription management, and order checkout — must function correctly according to defined requirements. Any module failing its expected outcome will be marked as **Fail**.

2. **Data Accuracy & Integrity**: User-related data (e.g., cart contents, refill schedules, account preferences) must be stored and retrieved without errors. Any mismatched or lost data during operations will result in a **Fail**.

3. **System Stability & Error Handling**: The system must operate without crashes or critical errors during normal use. Smooth navigation and proper error feedback are mandatory. Unhandled exceptions or user-facing bugs will be treated as **Fail** conditions.

4. **Performance Benchmarks**: The application should handle high-traffic situations such as flash sales or peak hours without slowing down. If page load times or order processing exceeds the acceptable threshold (e.g., 2 seconds), it will be marked as **Fail**.

5. **User Acceptance & Expectations**: All features must align with user requirements, including a seamless shopping experience, timely refill suggestions, and accurate product bundling. Any deviation from user expectations as defined in acceptance criteria will lead to a **Fail**.

6. **Accessibility & Usability**: The interface must be intuitive for a wide range of users. If any user struggles with navigation, text readability, or overall experience due to poor UI/UX or lack of accessibility features, it will result in a **Fail**.

Out of the **XX** test cases executed for the AutoCart project, **YY** test cases have passed, while **ZZ** encountered issues. This results in a pass rate of approximately **AA%** and a fail rate of **BB%**. Despite a few failed cases, the high success rate indicates a stable and well-performing system. Fixing the failed items will further enhance the robustness and user satisfaction of AutoCart.

10. TEST DELIVERABLES

A. **Test Plan**

A comprehensive document outlining the overall testing strategy, objectives, scope, testing environment, and schedule. It includes test coverage for AutoCart features like AI-powered refill prediction, smart bundling, and subscription management.

B. **Test Cases**

A collection of detailed test cases that define test inputs, execution conditions, expected outcomes, and pass/fail criteria. These include scenarios like automated refill triggers, personalized bundle generation, and payment gateway validation.

C. **Test Execution Logs**

Logs recording the results of executed test cases, including timestamps, outcomes (pass/fail), issues encountered, and relevant screenshots or logs. These help in auditing and issue traceability.

D. **Defect Reports**

Structured documentation of all bugs and issues identified during testing. Each report includes the defect ID, description, severity, steps to reproduce, and resolution status to assist developers in fixing the issues efficiently.

E. **Test Summary Report**
A final report summarizing testing outcomes, highlighting the number of test cases executed, success rates, unresolved bugs, and overall system readiness for deployment.

F. **Screen Prototypes & UI Mockups**
Visual layouts and interaction models of major screens such as the cart interface, refill suggestions, subscription control, and checkout flow, which were used as references during testing.

G. **Turnover Documentation**
A formal document handed over at the end of the testing phase, detailing test completion status, critical issues (if any), and transition notes for deployment and maintenance teams.

11. STAFFING AND TRAINING NEEDS

Staffing Requirements

- **1 Full-Time Tester:** Dedicated to system, integration, and acceptance testing phases. Initially involved part-time during early development and assigned full-time approximately halfway through the project. If a dedicated tester is unavailable, the project manager or test lead will take on testing responsibilities.
- **2 AI/ML Engineers:** Support testing related to AI-powered features such as refill prediction and smart bundling.
- **2 Backend Developers:** Assist with unit testing, bug fixing, and integration testing.
- **2 Frontend Developers:** Support UI testing and usability verification.
- **Project Manager/Test Lead:** Oversees testing activities, defect management, and ensures testing quality and schedule adherence.

Training Requirements

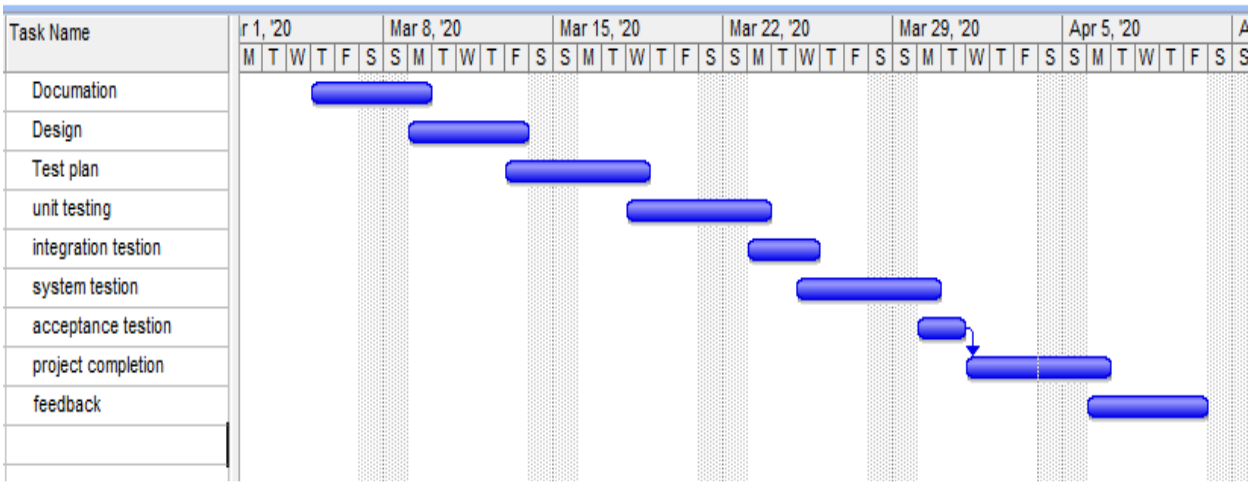
- **Testers and Developers:** Training on AutoCart's key features, including AI refill prediction models, subscription management workflows, smart bundling logic, and payment integration.
- **Operations Staff:** Training on monitoring AI model outputs, managing subscription adjustments, and handling customer service workflows.
- **Support and Sales Teams:** Training to assist users with subscription management, auto-refill settings, and troubleshooting common issues.
- **Security Team:** Training on implementing and monitoring multi-factor authentication, data privacy regulations, and secure transaction processing.

12. RESPONSIBILITIES

	TM	PM	Dev Team	Test Team	Client
Acceptance test Documentation & Execution	X	X		X	X
System/Integration test Documentation & Exec.	X		X	X	
Unit test documentation & execution	X		X	X	
System Design Reviews	X	X	X	X	X
Detail Design Reviews	X	X	X	X	
Test procedures and rules	X	X	X	X	
Screen & Report prototype reviews			X	X	X
Change Control and regression testing	X	X	X	X	X

13. TESTING SCHEDULE

Time has been allocated within the project plan for the following testing activities. The specific dates and times for each activity are defined in the project plan timeline. The persons required for each process are detailed in the project timeline and plan as well. Coordination of the personnel required for each task, test team, development team, management and customer will be handled by the project manager in conjunction with the development and test team leaders. Schedule must be done using any project management tool. Don't make the schedule using MS word/ excel.



14. PLANNING RISKS AND CONTINGENCIES

- **Limited Availability of Domain Experts:** Experts in e-commerce, subscription management, or AI may have restricted availability, potentially delaying validation of predictive models and feature usability. To mitigate this, flexible scheduling and backup consultants will be arranged.

- **Data Privacy and Security Risks:** Handling customer purchase data and subscription details involves privacy concerns. To address this, strong data encryption, anonymization techniques, and frequent security audits will be implemented.

- **Third-Party API Integration Delays:** Integration with external payment gateways, delivery services, or AI service providers may face delays. Early API testing, proactive communication with vendors, and use of mock APIs will help minimize impact.

- **Limited Testing Resources:** Constraints in QA personnel or tools could affect testing timelines. Cross-training team members and prioritizing critical test cases will help optimize resource utilization.

- **User Acceptance Testing (UAT) Delays:** Delays in customer or stakeholder feedback during UAT could impact project deadlines. A well-defined UAT schedule with regular follow-ups and clear communication channels will be maintained.

- **Regulatory and Compliance Challenges:** Failure to meet e-commerce and data protection regulations may delay launch. Regular compliance checks and consultations with legal experts will be conducted throughout the project.

15. APROVALS

Project Sponsor - Steve Sponsor	
Development Management - Ron Manager	
EDI Project Manager - Peggy Project	
RS Test Manager - Dale Tester	
RS Development Team Manager - Dale Tester	
Reassigned Sales - Cathy Sales	
Order Entry EDI Team Manager - Julie Order	